

LAPORAN PRATIKUM STRUKTUR DATA
“BINARY TREE DAN GRAPH”

disusun Oleh:

AFIF RAHMAN SALEH

NIM 251153105

Dosen Pengampu: Dr. WAHYUDI, S.T, M.T

Asisten Praktikum : M Galid Alvero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

TAHUN

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas rahmat-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan. Laporan ini disusun sebagai bentuk pertanggungjawaban kegiatan praktikum pekan pertama dengan topik implementasi Stack pada java.

Ucapan terima kasih penulis sampaikan kepada dosen pengampu, asisten praktikum, serta rekan-rekan mahasiswa atas bimbingan dan kerja sama yang diberikan. Penulis menyadari laporan ini masih jauh dari sempurna, sehingga kritik dan saran sangat diharapkan demi perbaikan di masa mendatang.

Semoga laporan ini bermanfaat bagi penulis maupun pembaca dalam memahami konsep struktur data dan penerapannya dalam pemrograman.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	3
2.1 Landasan Teori	3
2.2 Kode Java yang Dipraktikkan	3
2.3 Penjelasan Kode Program	9
2.4 Analisis Hasil	14
BAB III KESIMPULAN.....	16
3.1 Kesimpulan Praktikum.....	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan fondasi utama dalam pengembangan perangkat lunak yang efisien. Selain struktur data linear seperti array, linked list, stack, dan queue, terdapat struktur data non-linear yang sangat penting untuk merepresentasikan data yang memiliki hubungan hierarkis maupun jaringan, yaitu *Tree* (Pohon) dan *Graph* (Graf).

Binary Tree sering digunakan untuk merepresentasikan hierarki data, struktur folder, atau ekspresi aritmatika. Sementara itu, Graph digunakan untuk memodelkan jaringan sosial, peta rute, dan topologi jaringan komputer. Pemahaman mengenai cara mengimplementasikan, memanipulasi, dan menelusuri (traversal) struktur data ini menggunakan bahasa pemrograman Java sangat diperlukan untuk menyelesaikan berbagai masalah komputasi yang kompleks. Oleh karena itu, praktikum ini dilakukan untuk mengimplementasikan konsep Binary Tree dan Graph Traversal (DFS dan BFS) secara langsung.

1.2 Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dasar struktur data non-linear, khususnya Binary Tree dan Graph.
2. Mahasiswa mampu mengimplementasikan Binary Tree beserta operasi dasarnya (penambahan node, pencarian, dan penghitungan jumlah node) menggunakan Java.
3. Mahasiswa mampu menerapkan algoritma penelusuran Binary Tree (Preorder, Inorder, Postorder).
4. Mahasiswa mampu mengimplementasikan representasi Graph menggunakan Adjacency List.
5. Mahasiswa mampu menerapkan dan menganalisis algoritma penelusuran Graph yaitu Depth-First Search (DFS) dan Breadth-First Search (BFS).

1.3 Manfaat Praktikum

1. Meningkatkan kemampuan pemecahan masalah (problem solving) terkait struktur data non-linear.
2. Memberikan pemahaman praktis mengenai bagaimana data hierarkis dan relasional dimodelkan dalam program.
3. Melatih penggunaan konsep rekursif (recursion) dan Collection Framework (seperti HashMap, ArrayList, Queue, Set) dalam Java.

BAB II

PEMBAHASAN

2.1 Landasan Teori

1. Binary Tree (Pohon Biner)

Binary Tree adalah struktur data non-linear yang terdiri dari Kumpulan node yang di mana setiap node maksimal memiliki dua anak/cabang, yaitu right child dan left child. Node paling atas disebut dengan root. Penelusuran (tranversal) pada binary tree ada tiga macam, yaitu :

- a. Inorder (left – root – right) : menelusuri subtree kiri, mengunjungi root, dan menelusuri subtree kanan.
- b. Preorder (root – left – right): mengunjungi root, menelusuri subtree kiri, dan menelusuri subtree kanan.
- c. Postorder (left – right – root) : menelusuri subtree kiri, menelusuri subtree kanan, mengunjungi root.

2. Graph (Graf)

Graph adalah struktur data yang terdiri dari *Vertex* (simpul/node) dan *Edge* (sisi/penghubung). Graph dapat direpresentasikan menggunakan *Adjacency List* (Daftar Ketetanggaan) yang memanfaatkan *HashMap* dan *ArrayList*.

- a. DFS (Depth -First Search) : Penelusuran graf yang menelusuri sejauh mungkin ke dalam (kedalaman) sebelum melakukan *backtracking*. Biasanya diimplementasikan menggunakan rekursi atau *Stack*.
- b. BFS (Breadth-First Search): Penelusuran graf yang menelusuri tetangga terdekat terlebih dahulu (melebar/level by level). Diimplementasikan menggunakan *Queue*.

2.2 Kode Java yang Dipraktikkan

1. Kelas Node_2511531005

```

• package pekan9_2511531005;
•
• public class Node_2511531005 {
•     int data_1005;
•     Node_2511531005 left_1005;
•     Node_2511531005 right_1005;
•
•     public Node_2511531005(int data_1005) {
•         this.data_1005 = data_1005;
•         left_1005 = null;
•         right_1005 = null;
•     }
•
•     public void setLeft_2511531005(Node_2511531005 node_1005) {
•         if (left_1005 == null)
•             left_1005 = node_1005;
•     }
•
•     public void setRight_2511531005(Node_2511531005 node_1005) {
•         if (right_1005 == null)
•             right_1005 = node_1005;
•     }
•
•     public Node_2511531005 getLeft_1005() {
•         return left_1005;
•     }
•
•     public Node_2511531005 getRight_1005() {
•         return right_1005;
•     }
•
•     public int getData_1005() {
•         return data_1005;
•     }
•
•     public void setData_1005(int data_1005) {
•         this.data_1005 = data_1005;
•     }
•
•     void printPreorder_1005(Node_2511531005 node_1005) {
•         if (node_1005 == null)
•             return;
•         System.out.print(node_1005.data_1005 + " ");
•         printPreorder_1005(node_1005.left_1005);
•         printPreorder_1005(node_1005.right_1005);
•     }
•
•     void printPostorder_1005(Node_2511531005 node_1005) {
•         if (node_1005 == null)
•             return;
•         printPostorder_1005(node_1005.left_1005);
•         printPostorder_1005(node_1005.right_1005);
•         System.out.print(node_1005.data_1005 + " ");
•     }
•
• }

```

```

• void printInorder_1005(Node_2511531005 node_1005) {
•     if (node_1005 == null)
•         return;
•     printInorder_1005(node_1005.left_1005);
•     System.out.print(node_1005.data_1005 + " ");
•     printInorder_1005(node_1005.right_1005);
• }
•
• public void print_2511531005() {
•     print_1005("", true, "");
• }
•
• public void print_1005(String prefix_1005, boolean isTail_1005, String
sb_1005) {
•     if (right_1005 != null) {
•         right_1005.print_1005(prefix_1005 + (isTail_1005 ? "| " : " "),
false, sb_1005);
•     }
•     System.out.println(prefix_1005 + (isTail_1005 ? "\\--" : "/--") +
data_1005);
•     if (left_1005 != null) {
•         left_1005.print_1005(prefix_1005 + (isTail_1005 ? " " : "| "),
true, sb_1005);
•     }
• }
• }
• }

```

2. Kelas BTree_2511531005

```

• package pekan9_2511531005;
•
• public class BTree_2511531005 {
•     private Node_2511531005 root_1005;
•     private Node_2511531005 currentNode_1005;
•
•     public BTree_2511531005() {
•         root_1005 = null;
•     }
•
•     public boolean search_1005(int data_1005) {
•         return search_1005(root_1005, data_1005);
•     }
•
•     private boolean search_1005(Node_2511531005 node_1005, int data_1005) {
•         if (node_1005 == null)
•             return false;
•         if (node_1005.getData_1005() == data_1005)
•             return true;
•         if (node_1005.getLeft_1005() != null)
•             if (search_1005(node_1005.getLeft_1005(), data_1005))
•                 return true;
•         if (node_1005.getRight_1005() != null)
•             if (search_1005(node_1005.getRight_1005(), data_1005))
•                 return true;
•     }
• }

```



```

    return false;
}

public void printInorder_1005() {
    if (root_1005 != null)
        root_1005.printInorder_1005(root_1005);
}

public void printPreorder_1005() {
    if (root_1005 != null)
        root_1005.printPreorder_1005(root_1005);
}

public void printPostorder_1005() {
    if (root_1005 != null)
        root_1005.printPostorder_1005(root_1005);
}

public Node_2511531005 getRoot_1005() {
    return root_1005;
}

public void setRoot_1005(Node_2511531005 root_1005) {
    this.root_1005 = root_1005;
}

public boolean isEmpty_1005() {
    return root_1005 == null;
}

public int countNodes_1005() {
    return countNodes_1005(root_1005);
}

private int countNodes_1005(Node_2511531005 node_1005) {
    if (node_1005 == null) {
        return 0;
    } else {
        int count_1005 = 1;
        count_1005 += countNodes_1005(node_1005.getLeft_1005());
        count_1005 += countNodes_1005(node_1005.getRight_1005());
        return count_1005;
    }
}

public void print() {
    if (root_1005 != null)
        root_1005.print_2511531005();
}

public Node_2511531005 getCurrent_1005() {
    return currentNode_1005;
}

public void setCurrent_1005(Node_2511531005 node_1005) {

```

```

    •         this.currentNode_1005 = node_1005;
    •     }
    • }

```

3. Kelas BTreeDriver_2511531005

```

    • package pekan9_2511531005;
    •
    • public class BtreeDriver_2511531005 {
    •
    •     public static void main(String[] args) {
    •         // membuat Pohon
    •         BTree_2511531005 tree_1005 = new BTree_2511531005();
    •         System.out.print("Jumlah Simpul awal pohon: ");
    •         System.out.println(tree_1005.countNodes_1005());
    •
    •         // menambahkan simpul data 1
    •         Node_2511531005 root_1005 = new Node_2511531005(1);
    •
    •         // menjadikan simpul 1 sebagai root
    •         tree_1005.setRoot_1005(root_1005);
    •         System.out.println("Jumlah simpul jika hanya ada root");
    •         System.out.println(tree_1005.countNodes_1005());
    •
    •         Node_2511531005 node2 = new Node_2511531005(2);
    •         Node_2511531005 node3 = new Node_2511531005(3);
    •         Node_2511531005 node4 = new Node_2511531005(4);
    •         Node_2511531005 node5 = new Node_2511531005(5);
    •         Node_2511531005 node6 = new Node_2511531005(6);
    •         Node_2511531005 node7 = new Node_2511531005(7);
    •         Node_2511531005 node8 = new Node_2511531005(8);
    •         Node_2511531005 node9 = new Node_2511531005(9);
    •
    •         root_1005.setLeft_2511531005(node2);
    •         node2.setLeft_2511531005(node4);
    •         node2.setRight_2511531005(node5);
    •         node4.setRight_2511531005(node8);
    •         root_1005.setRight_2511531005(node3);
    •         node3.setLeft_2511531005(node6);
    •         node3.setRight_2511531005(node7);
    •         node6.setLeft_2511531005(node9);
    •
    •         // set current node
    •         tree_1005.setCurrent_1005(tree_1005.getRoot_1005());
    •         System.out.println("menampilkan simpul terakhir: ");
    •         System.out.println(tree_1005.getCurrent_1005().getData_1005());
    •
    •         System.out.println("jumlah simpul setelah simpul ditambahkan ");
    •         System.out.println(tree_1005.countNodes_1005());
    •
    •         System.out.println("InOrder: ");
    •         tree_1005.printInorder_1005();
    •
    •     }
    • }

```

```

•         System.out.println("\nPreOrder: ");
•         tree_1005.printPreorder_1005();
•
•         System.out.println("\nPostOrder: ");
•         tree_1005.printPostorder_1005();
•
•         System.out.println("\nmenampilkan simpul dalam bentuk pohon");
•         tree_1005.print();
•     }
• }

```

4. Kelas GraphTranversal_2511531005

```

• package pekan9_2511531005;
• import java.util.*;
•
• public class GraphTraversal_2511531005 {
•
•     private Map<String, List<String>> graph_1005 = new HashMap<>();
•
•     public void addEdge_1005(String node1_1005, String node2_1005) {
•         graph_1005.putIfAbsent(node1_1005, new ArrayList<>());
•         graph_1005.putIfAbsent(node2_1005, new ArrayList<>());
•         graph_1005.get(node1_1005).add(node2_1005);
•         graph_1005.get(node2_1005).add(node1_1005);
•
•     }
•
•     public void printGraph_1005() {
•         System.out.println("graf awal (adjacency list) : ");
•         for(String node_1005 : graph_1005.keySet()) {
•             System.out.print(node_1005 + " -> ");
•             List<String> neighbors_1005 = graph_1005.get(node_1005);
•             System.out.println(String.join(", ", neighbors_1005));
•         }
•         System.out.println();
•     }
•
•     public void dfs_1005(String start_1005) {
•         Set<String> visited_1005 = new HashSet<>();
•         System.out.println("penelusuran DFS : ");
•         dfsHelper_1005(start_1005, visited_1005);
•         System.out.println();
•     }
•
•     public void dfsHelper_1005(String current_1005, Set<String> visited_1005) {
•         if(visited_1005.contains(current_1005)) return;
•         visited_1005.add(current_1005);
•         System.out.print(current_1005 + " ");
•         for(String neighbor_1005 : graph_1005.getDefault(current_1005, new
• ArrayList<>())) {
•             dfsHelper_1005(neighbor_1005, visited_1005);
•         }
•     }
• }

```

```

•
•
• public void bfs_1005(String start_1005) {
•
•     Set<String> visited_1005 = new HashSet<>();
•     Queue<String> queue_1005 = new LinkedList<>();
•     queue_1005.add(start_1005);
•     visited_1005.add(start_1005);
•
•
•     System.out.println("penelusuran BFS:");
•     while(!queue_1005.isEmpty()) {
•         String current_1005 = queue_1005.poll();
•         System.out.print(current_1005 + " ");
•         for (String neighbor_1005 : graph_1005.getDefault(current_1005, new
• ArrayList<>())) {
•             if(!visited_1005.contains(neighbor_1005)) {
•                 queue_1005.add(neighbor_1005);
•                 visited_1005.add(neighbor_1005);
•             }
•         }
•     }
•     System.out.println();
•
• }
•
• public static void main (String[] args) {
•
•     GraphTraversal_2511531005 graphObj_1005 = new GraphTraversal_2511531005
• ();
•
•     graphObj_1005.addEdge_1005("A", "B");
•     graphObj_1005.addEdge_1005("A", "C");
•     graphObj_1005.addEdge_1005("B", "D");
•     graphObj_1005.addEdge_1005("B", "E");
•
•     System.out.println("graf awal adalah :");
•     graphObj_1005.printGraph_1005();
•
•     graphObj_1005.dfs_1005("A");
•     graphObj_1005.bfs_1005("A");
•
• }
•
• }

```

2.3 Penjelasan Kode Program

1. Kelas Node_2511531005

Kelas ini berfungsi sebagai cetak biru untuk setiap simpul pada binary tree. Logikanya adalah untuk menyimpan data dan mereferensikan alamat memori untuk anak kiri dan anak kanannya. Selain itu kelas ini terdapat logika rekursif yaitu

preorder, inorder, dan postorder. Serta logika pencetakan visual untuk menggambar struktur pohon. Berikut langkah-langkahnya:

1. Deklarasi kelas dan atribut

```
public class Node_2511531005 {  
    int data_1005;  
    Node_2511531005 left_1005;  
    Node_2511531005 right_1005;  
}
```

2. Konstruktor

```
public Node_2511531005(int data_1005) {  
    this.data_1005 = data_1005;  
    left_1005 = null;  
    right_1005 = null;  
}
```

3. Setter anak kiri dan kanan

```
public void setLeft_2511531005(Node_2511531005 node_1005) {  
    if (left_1005 == null)  
        left_1005 = node_1005;  
}  
  
public void setRight_2511531005(Node_2511531005 node_1005) {  
    if (right_1005 == null)  
        right_1005 = node_1005;  
}
```

4. Getter dan setter data

```
public Node_2511531005 getLeft_1005() {  
    return left_1005;  
}  
  
public Node_2511531005 getRight_1005() {  
    return right_1005;  
}  
  
public int getData_1005() {  
    return data_1005;  
}  
  
public void setData_1005(int data_1005) {  
    this.data_1005 = data_1005;  
}
```

5. Traversal tree

- a. Preorder (root – left – root)

```
void printPreorder_1005(Node_2511531005 node_1005) {
    if (node_1005 == null)
        return;
    System.out.print(node_1005.data_1005 + " ");
    printPreorder_1005(node_1005.left_1005);
    printPreorder_1005(node_1005.right_1005);
}
```

b. Postorder (left – right – root)

```
void printPostorder_1005(Node_2511531005 node_1005) {
    if (node_1005 == null)
        return;
    printPostorder_1005(node_1005.left_1005);
    printPostorder_1005(node_1005.right_1005);
    System.out.print(node_1005.data_1005 + " ");
}
```

c. Inorder (left – root – right)

```
void printInorder_1005(Node_2511531005 node_1005) {
    if (node_1005 == null)
        return;
    printInorder_1005(node_1005.left_1005);
    System.out.print(node_1005.data_1005 + " ");
    printInorder_1005(node_1005.right_1005);
}
```

6. Visualisasi tree

```
public void print_2511531005() {
    print_1005("", true, "");
}
```

2. Kelas BTree_2511531005

Kelas ini berfungsi sebagai pengelola struktur binary tree. Berikut langkah-langkahnya :

1. Deklarasi atribut

```
private Node_2511531005 root_1005;
private Node_2511531005 currentNode_1005;
```

root sebagai node akar dari tree dan currentnode sebagai node yang sedang aktif

2. Konstruktor

```
public BTree_2511531005() {  
    root_1005 = null;  
}
```

3. Pencarian data

```
public boolean search_1005(int data_1005) {  
    return search_1005(root_1005, data_1005);  
}  
  
private boolean search_1005(Node_2511531005 node_1005, int data_1005) {  
    if (node_1005 == null)  
        return false;  
    if (node_1005.getData_1005() == data_1005)  
        return true;  
    if (node_1005.getLeft_1005() != null)  
        if (search_1005(node_1005.getLeft_1005(), data_1005))  
            return true;  
    if (node_1005.getRight_1005() != null)  
        if (search_1005(node_1005.getRight_1005(), data_1005))  
            return true;  
    return false;  
}
```

4. Traversal tree

Menampilkan isi tree sesuai urutan

5. Getter dan setter root

Untuk menganbil atau menetapkan node akar tree

6. Cek kosong

Mengembalikan true jika tree belum punya root

7. Menghitung jumlah node

Memanggil fungsi rekursif untuk menghitung node

8. Print visual tree

```
public void print() {  
    if (root_1005 != null)  
        root_1005.print_2511531005();  
}
```

9. Current node

```
public Node_2511531005 getCurrent_1005() {  
    return currentNode_1005;  
}
```

```
public void setCurrent_1005(Node_2511531005 node_1005) {
    this.currentNode_1005 = node_1005;
}
```

Menyimpan node yang sedang aktif

3. Kelas BTreeDriver_2511531005

Kelas ini berfungsi untuk menguji kelas BTree dan Node tadi. Berikut penjelasannya :

1. countNode_1005()
digunakan untuk menghitung jumlah node dalam tree
2. setRoot_1005(Node root)
untuk menetapkan node yang dibuat sebagai akar tree
3. setLeft dan setRight_2511531005(Node node)
untuk menambahkan anak kiri dan anak kanan pada node tertentu
4. setCurrent dan getCurrent
5. printPreorder
6. printPostorder
7. prinInorder
8. print()
menampilkan tree dalam bentuk diagram bercabang

4. Kelas GraphTranversal_2511531005

Berikut penjelasan langkah langkahnya :

1. addEdge_1005()
untuk menambahkan edge dua arah.
2. printGraph_1005()
untuk menampilkan adjacency list.
3. dfs_1005() + dfsHelper_1005()
penelusuran graph dengan DFS (rekursif).
4. bfs_1005()
penelusuran graph dengan BFS (queue).

5. main()

membangun graph, menampilkan isi, lalu menjalankan DFS & BFS.

2.4 Analisis Hasil

Berdasarkan program pada semua kelas yang dipraktikkan pada pekan ini, diperoleh beberapa hasil sebagai berikut:

1. Analisis hasil binary tree

Dari kelas

Output :

```
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
jumlah simpul setelah simpul ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
| /--7
| /--3
| | \--6
| | \--9
|--1
| /--5
|--2
| /--8
|--4
```

Berdasarkan output di atas, dapat disimpulkan bahwa struktur *Binary Tree* berhasil dibentuk dan dirangkai dengan benar yang terdiri dari 9 simpul dengan

simpul 1 sebagai akar (*root*). Perbedaan urutan angka pada *Inorder*, *Preorder*, dan *Postorder* membuktikan bahwa logika rekursif untuk penelusuran pohon bekerja sesuai dengan prinsip masing-masing (kiri-akar-kanan, akar-kiri-kanan, dan kiri-kanan-akar). Selain itu, visualisasi pohon yang dicetak secara menyamping berhasil merepresentasikan hierarki data dengan tepat, di mana anak kanan berada di atas dan anak kiri berada di bawah dari simpul induknya.

2. Kelas GraphTranversal_2511531005

Output :

```
graf awal adalah :  
graf awal (adjacency list) :  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
penelusuran DFS :  
A B D E C  
penelusuran BFS:  
A B C D E
```

Berdasarkan output di atas, dapat disimpulkan bahwa representasi Graph menggunakan Adjacency List berhasil memetakan hubungan ketetanggaan antar node secara dua arah (undirected) dengan tepat. Perbedaan hasil penelusuran membuktikan karakteristik kedua algoritma: DFS (Depth-First Search) menelusuri graf secara mendalam (menukik ke dalam) hingga mentok baru melakukan backtracking (A-B-D-E-C), sedangkan BFS (Breadth-First Search) menelusuri graf secara melebar (level by level) berdasarkan kedekatan jarak dari node awal (A-B-C-D-E).

BAB III

KESIMPULAN

3.1 Kesimpulan Praktikum

Berdasarkan praktikum yang telah dilakukan mengenai implementasi Binary Tree dan Graph Traversal, dapat ditarik kesimpulan bahwa struktur data non-linear ini sangat efektif untuk merepresentasikan data yang memiliki hubungan hierarkis maupun relasional. Pada Binary Tree, pembentukan simpul dan operasi penelusurannya sangat bergantung pada penerapan fungsi rekursif, yang mana masing-masing metode (Inorder, Preorder, Postorder) menghasilkan urutan data yang berbeda tergantung pada waktu kunjungan ke simpul akar. Struktur pohon juga berhasil divisualisasikan dengan baik melalui logika pencetakan teks yang mengatur posisi anak kiri dan kanan secara spasial.

Sementara itu, pada struktur Graph, penggunaan Adjacency List melalui HashMap dan ArrayList terbukti efisien untuk menyimpan dan memetakan relasi ketetanggaan antar simpul. Penelusuran graf menggunakan Depth-First Search (DFS) dan Breadth-First Search (BFS) menunjukkan perbedaan pendekatan yang signifikan dalam menyelesaikan masalah graf. DFS menggunakan rekursi untuk menelusuri secara mendalam hingga ke ujung cabang, sedangkan BFS memanfaatkan struktur Queue untuk menelusuri secara melebar level demi level. Secara keseluruhan, praktikum ini menegaskan pentingnya pemahaman konsep rekursi dan penguasaan Collection Framework dalam Java untuk mengimplementasikan dan menyelesaikan permasalahan komputasi yang melibatkan struktur data kompleks.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Tree Traversals (Inorder, Preorder and Postorder)," *GeeksforGeeks*, 2023. [Online]. Tersedia: <https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>. [Diakses: 11-Jun-2026].
- [2] GeeksforGeeks, "Breadth First Search or BFS for a Graph," *GeeksforGeeks*, 2023. [Online]. Tersedia: <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>. [Diakses: 11-Jun-2026].
- [3] GeeksforGeeks, "Depth First Search or DFS for a Graph," *GeeksforGeeks*, 2023. [Online]. Tersedia: <https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>. [Diakses: 11-Jun-2026].
- [4] Oracle, "Java Platform, Standard Edition Documentation," *Oracle Help Center*, 2024. [Online]. Tersedia: <https://docs.oracle.com/en/java/javase/>. [Diakses: 11-Jun-2026].

TUGAS PRAKTIKUM PEKAN 9

○ Kode Program

```
package pekan9_2511531005;

import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Point;
import java.awt.RenderingHints;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import java.util.Stack;

import javax.swing.*;
import javax.swing.border.TitledBorder;

public class PetaPadang_2511531005 extends JFrame implements
ActionListener {

    private List<String> vertices_1005;
    private Map<String, List<String>> adjList_1005;
    private Map<String, Point> nodePosition_1005;

    private JComboBox<String> startCombo_1005;
    private JComboBox<String> goalCombo_1005;
    private JButton btnBFS_1005;
    private JButton btnDFS_1005;
    private JButton btnReset_1005;
    private JTextArea resultArea_1005;
    private GraphPanel_1005 canvas_1005;

    private List<String> visitedNodes_1005;
    private List<String> pathNodes_1005;
    private int exploredCount_1005;
    private String lastMethod_1005;
    private String lastStart_1005;
    private String lastGoal_1005;
```

```

public PetaPadang_2511531005() {
    setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
    setSize(900, 700);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(10, 10));

    initializeGraph_1005();

    visitedNodes_1005 = new ArrayList<>();
    pathNodes_1005 = new ArrayList<>();
    exploredCount_1005 = 0;
    lastMethod_1005 = "";
    lastStart_1005 = "";
    lastGoal_1005 = "";

    setupGUI_1005();
}

//inisialisasi graf
public void initializeGraph_1005() {
    vertices_1005 = new ArrayList<>();
    adjList_1005 = new HashMap<>();
    nodePosition_1005 = new HashMap<>();

    String[] locations_1005 = {
        "Lubuk Buayo", "Unand", "Spendu", "Katib",
        "Jati", "Pondok", "Taplau", "Basko", "Gacoan bypass",
        "Kopkit sutomo"
    };

    for(String loc_1005 : locations_1005) {
        vertices_1005.add(loc_1005);
        adjList_1005.put(loc_1005, new ArrayList<>());
    }

    // Koordinat Node (X, Y)
    nodePosition_1005.put("Unand", new Point(500, 20));
    nodePosition_1005.put("Lubuk Buayo", new Point(200, 120));
    nodePosition_1005.put("Spendu", new Point(800, 120));
    nodePosition_1005.put("Katib", new Point(200, 220));
    nodePosition_1005.put("Jati", new Point(500, 220));
    nodePosition_1005.put("Pondok", new Point(800, 220));
    nodePosition_1005.put("Taplau", new Point(500, 320));
    nodePosition_1005.put("Basko", new Point(200, 420));
    nodePosition_1005.put("Gacoan bypass", new Point(500, 420));
    nodePosition_1005.put("Kopkit sutomo", new Point(800, 420));

    // 15 Edge Penghubung
    addEdge_1005("Lubuk Buayo", "Unand");
    addEdge_1005("Lubuk Buayo", "Spendu");
    addEdge_1005("Unand", "Spendu");
    addEdge_1005("Pondok", "Spendu");
    addEdge_1005("Lubuk Buayo", "Jati");
    addEdge_1005("Lubuk Buayo", "Katib");
    addEdge_1005("Gacoan bypass", "Jati");

```

```

        addEdge_1005("Jati", "Pondok");
        addEdge_1005("Pondok", "Kopkit sutomo");
        addEdge_1005("Jati", "Kopkit sutomo");
        addEdge_1005("Kopkit sutomo", "Basko");
        addEdge_1005("Basko", "Taplau");
        addEdge_1005("Basko", "Katib");
        addEdge_1005("Taplau", "Katib");
        addEdge_1005("Jati", "Taplau");
    }

    private void addEdge_1005(String u_1005, String v_1005) {
        adjList_1005.get(u_1005).add(v_1005);
        adjList_1005.get(v_1005).add(u_1005);
    }

    //setup gui
    public void setupGUI_1005() {
        // Panel Kontrol Atas
        JPanel controlPanel_1005 = new JPanel(new
FlowLayout(FlowLayout.LEFT, 20, 15));
        controlPanel_1005.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.GRAY, 1),
            "Pencarian Jalur Menggunakan BFS dan DFS",
            TitledBorder.LEFT,
            TitledBorder.TOP,
            new Font("Arial", Font.BOLD, 14)
        ));

        // Lokasi Awal
        JLabel lblStart_1005 = new JLabel("Lokasi Awal : ");
        lblStart_1005.setFont(new Font("Arial", Font.PLAIN, 14));
        controlPanel_1005.add(lblStart_1005);

        startCombo_1005 = new JComboBox<>(vertices_1005.toArray(new
String[0]));
        startCombo_1005.setFont(new Font("Arial", Font.PLAIN, 14));
        controlPanel_1005.add(startCombo_1005);

        // Lokasi Tujuan
        JLabel lblGoal_1005 = new JLabel("Lokasi Tujuan : ");
        lblGoal_1005.setFont(new Font("Arial", Font.PLAIN, 14));
        controlPanel_1005.add(lblGoal_1005);

        goalCombo_1005 = new JComboBox<>(vertices_1005.toArray(new
String[0]));
        goalCombo_1005.setFont(new Font("Arial", Font.PLAIN, 14));
        goalCombo_1005.setSelectedIndex(9);
        controlPanel_1005.add(goalCombo_1005);

        // Tombol BFS, DFS, RESET
        btnBFS_1005 = new JButton("BFS");
        btnBFS_1005.setBackground(new Color(144, 238, 144));
        btnBFS_1005.setFont(new Font("Arial", Font.BOLD, 14));
        btnBFS_1005.setPreferredSize(new Dimension(90, 35));

        btnDFS_1005 = new JButton("DFS");
        btnDFS_1005.setBackground(new Color(255, 200, 120));
    }

```

```

        btnDFS_1005.setFont(new Font("Arial", Font.BOLD, 14));
        btnDFS_1005.setPreferredSize(new Dimension(90, 35));

        btnReset_1005 = new JButton("Reset");
        btnReset_1005.setBackground(new Color(255, 130, 130));
        btnReset_1005.setFont(new Font("Arial", Font.BOLD, 14));
        btnReset_1005.setPreferredSize(new Dimension(90, 35));

        btnBFS_1005.addActionListener(this);
        btnDFS_1005.addActionListener(this);
        btnReset_1005.addActionListener(this);

        controlPanel_1005.add(btnBFS_1005);
        controlPanel_1005.add(btnDFS_1005);
        controlPanel_1005.add(btnReset_1005);

        add(controlPanel_1005, BorderLayout.NORTH);

        // Panel Tengah (Visualisasi Graph)
        JPanel graphContainer_1005 = new JPanel(new BorderLayout());
        graphContainer_1005.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.GRAY, 1),
            "Visualisasi Graph",
            TitledBorder.LEFT,
            TitledBorder.TOP,
            new Font("Arial", Font.BOLD, 14)
        ));
        canvas_1005 = new GraphPanel_1005();
        graphContainer_1005.add(canvas_1005, BorderLayout.CENTER);

        add(graphContainer_1005, BorderLayout.CENTER);

        // Panel Bawah (Hasil Pencarian)
        JPanel resultPanel_1005 = new JPanel(new BorderLayout());
        resultPanel_1005.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.GRAY, 1),
            "Hasil Pencarian",
            TitledBorder.LEFT,
            TitledBorder.TOP,
            new Font("Arial", Font.BOLD, 14)
        ));
        resultArea_1005 = new JTextArea(8, 50);
        resultArea_1005.setEditable(false);
        resultArea_1005.setFont(new Font("Monospaced", Font.PLAIN, 13));
        resultArea_1005.setText(buildEmptyResult_1005());

        JScrollPane scrollPane_1005 = new JScrollPane(resultArea_1005);
        resultPanel_1005.add(scrollPane_1005, BorderLayout.CENTER);

        add(resultPanel_1005, BorderLayout.SOUTH);
    }

    private String buildEmptyResult_1005() {
        return "Algoritma\t\t: -\n" +
            "Lokasi Awal\t\t: -\n" +
            "Lokasi Tujuan\t\t: -\n" +

```



```

-----\n" +
        "Node Dikunjungi\t: -\n" +
        "Jumlah Dieksplorasi\t: 0 node\n" +
        "-----\n" +
        "Jalur\t\t\t: Belum ada pencarian";
    }

    //algoritma bfs
    public void BFS_1005(String start_1005, String goal_1005) {
        resetGraph_1005();

        Queue<String> queue_1005 = new LinkedList<>();
        Set<String> visitedSet_1005 = new LinkedHashSet<>();
        Map<String, String> parent_1005 = new HashMap<>();

        queue_1005.add(start_1005);
        visitedSet_1005.add(start_1005);

        boolean found_1005 = false;

        while(!queue_1005.isEmpty()) {
            String current_1005 = queue_1005.poll();
            visitedNodes_1005.add(current_1005);
            exploredCount_1005++;

            if(current_1005.equals(goal_1005)) {
                found_1005 = true;
                break;
            }

            List<String> neighbors_1005 = new
ArrayList<>(adjList_1005.get(current_1005));
            Collections.sort(neighbors_1005);

            for(String neighbor_1005 : neighbors_1005) {
                if(!visitedSet_1005.contains(neighbor_1005)) {
                    visitedSet_1005.add(neighbor_1005);
                    parent_1005.put(neighbor_1005, current_1005);
                    queue_1005.add(neighbor_1005);
                }
            }
        }

        if(found_1005) {
            reconstructPath_1005(parent_1005, start_1005, goal_1005);
        }
        lastMethod_1005 = "BFS";
        lastStart_1005 = start_1005;
        lastGoal_1005 = goal_1005;
        displayPath_1005("BFS");
    }

    //algoritma dfs
    public void DFS_1005(String start_1005, String goal_1005) {
        resetGraph_1005();
    }

```

```

Stack<String> stack_1005 = new Stack<>();
Set<String> visitedSet_1005 = new LinkedHashSet<>();
Map<String, String> parent_1005 = new HashMap<>();

stack_1005.push(start_1005);

boolean found_1005 = false;

while (!stack_1005.isEmpty()) {
    String current_1005 = stack_1005.pop();

    if (!visitedSet_1005.contains(current_1005)) {
        visitedSet_1005.add(current_1005);
        visitedNodes_1005.add(current_1005);
        exploredCount_1005++;

        if (current_1005.equals(goal_1005)) {
            found_1005 = true;
            break;
        }

        List<String> neighbors_1005 = new
ArrayList<>(adjList_1005.get(current_1005));
        Collections.sort(neighbors_1005);
        Collections.reverse(neighbors_1005);

        for (String neighbor_1005 : neighbors_1005) {
            if (!visitedSet_1005.contains(neighbor_1005)) {
                parent_1005.put(neighbor_1005, current_1005);
                stack_1005.push(neighbor_1005);
            }
        }
    }
}

if (found_1005) {
    reconstructPath_1005(parent_1005, start_1005, goal_1005);
}
lastMethod_1005 = "DFS";
lastStart_1005 = start_1005;
lastGoal_1005 = goal_1005;
displayPath_1005("DFS");
}

private void reconstructPath_1005(Map<String, String> parent_1005,
String start_1005, String goal_1005) {
    List<String> path_1005 = new ArrayList<>();
    String current_1005 = goal_1005;

    while (current_1005 != null) {
        path_1005.add(current_1005);
        current_1005 = parent_1005.get(current_1005);
    }
    Collections.reverse(path_1005);
    pathNodes_1005 = path_1005;
}

```

```

//display
public void displayGraph_1005(Graphics g_1005) {
    canvas_1005.repaint();
}

public void displayPath_1005(String method_1005) {
    StringBuilder sb_1005 = new StringBuilder();

    sb_1005.append("Algoritma\t\t:");
    sb_1005.append(method_1005).append("\n");
    sb_1005.append("Lokasi Awal\t\t:");
    sb_1005.append(getShortName_1005(lastStart_1005)).append("\n");
    sb_1005.append("Lokasi Tujuan\t\t:");
    sb_1005.append(getShortName_1005(lastGoal_1005)).append("\n");
    sb_1005.append("-----\n");

    List<String> visitedShort_1005 = new ArrayList<>();
    for (String n_1005 : visitedNodes_1005) {
        visitedShort_1005.add(getShortName_1005(n_1005));
    }
    sb_1005.append("Node Dikunjungi\t: ").append(String.join(" -> ",
visitedShort_1005)).append("\n");
    sb_1005.append("Jumlah Dieksplorasi\t:");
    sb_1005.append(exploredCount_1005).append(" node\n");
    sb_1005.append("-----\n");

    if (pathNodes_1005.isEmpty()) {
        sb_1005.append("Jalur\t\t\t: Tidak ditemukan!");
    } else {
        List<String> pathShort_1005 = new ArrayList<>();
        for (String n_1005 : pathNodes_1005) {
            pathShort_1005.add(getShortName_1005(n_1005));
        }
        sb_1005.append("Jalur\t\t\t: ").append(String.join(" -> ",
pathShort_1005));
    }

    resultArea_1005.setText(sb_1005.toString());
    displayGraph_1005(canvas_1005.getGraphics());
}

public void resetGraph_1005() {
    visitedNodes_1005.clear();
    pathNodes_1005.clear();
    exploredCount_1005 = 0;
    resultArea_1005.setText(buildEmptyResult_1005());
    canvas_1005.repaint();
}

private String getShortName_1005(String name_1005) {
    if (name_1005 == null) return "-";
    if (name_1005.contains("Unand")) return "Unand";
    if (name_1005.contains("Lubuk Buayo")) return "Lubuk Buayo";
    if (name_1005.contains("Spendu")) return "Spendu";
}

```

```

        if (name_1005.contains("Katib")) return "Katib";
        if (name_1005.contains("Jati")) return "Jati";
        if (name_1005.contains("Pondok")) return "Pondok";
        if (name_1005.contains("Taplau")) return "Taplau";
        if (name_1005.contains("Basko")) return "Basko";
        if (name_1005.contains("Gacoan bypass")) return "Gacoan Bypass";
        if (name_1005.contains("Kopkit sutomo")) return "Kopkit Sutomo";
        return name_1005;
    }

    @Override
    public void actionPerformed(ActionEvent e_1005) {
        String start_1005 = (String) startCombo_1005.getSelectedItem();
        String goal_1005 = (String) goalCombo_1005.getSelectedItem();

        if (e_1005.getSource() == btnReset_1005) {
            resetGraph_1005();
            return;
        }

        if (start_1005.equals(goal_1005)) {
            JOptionPane.showMessageDialog(this, "Lokasi awal dan tujuan tidak boleh sama!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        if (e_1005.getSource() == btnBFS_1005) {
            BFS_1005(start_1005, goal_1005);
        } else if (e_1005.getSource() == btnDFS_1005) {
            DFS_1005(start_1005, goal_1005);
        }
    }

    class GraphPanel_1005 extends JPanel {

        // Warna sesuai legenda
        private final Color COLOR_START_1005 = new Color(0, 230, 0);
        // Hijau - Start Node
        private final Color COLOR_GOAL_1005 = new Color(230, 0, 0);
        // Merah - Goal Node
        private final Color COLOR_PATH_EDGE_1005 = new Color(255, 200, 0);
        // Kuning - Edge Jalur
        private final Color COLOR_VISITED_1005 = new Color(0, 220, 220);
        // Cyan - Dikunjungi
        private final Color COLOR_UNVISITED_1005 = new Color(190, 190, 190);
        // Abu-abu - Belum dikunjungi
        private final Color COLOR_TEXT_DARK_1005 = Color.BLACK;
        private final Color COLOR_TEXT_LIGHT_1005 = Color.WHITE;

        private final int NODE_RADIUS_1005 = 40;

        @Override
        protected void paintComponent(Graphics g_1005) {
            super.paintComponent(g_1005);
            Graphics2D g2d_1005 = (Graphics2D) g_1005;

```

```

        g2d_1005.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        String startSel_1005 = (String)
startCombo_1005.getSelectedItem();
        String goalSel_1005 = (String)
goalCombo_1005.getSelectedItem();

        // 1. Gambar semua Edge (garis abu-abu tipis)
g2d_1005.setColor(new Color(120, 120, 120));
g2d_1005.setStroke(new BasicStroke(1.5f));
Set<String> drawnEdges_1005 = new LinkedHashSet<>();
for (String u_1005 : adjList_1005.keySet()) {
    Point p1_1005 = nodePosition_1005.get(u_1005);
    for (String v_1005 : adjList_1005.get(u_1005)) {
        String key_1005 = u_1005.compareTo(v_1005) < 0 ?
u_1005 + "|" + v_1005 : v_1005 + "|" + u_1005;
        if (drawnEdges_1005.contains(key_1005)) continue;
        drawnEdges_1005.add(key_1005);
        Point p2_1005 = nodePosition_1005.get(v_1005);
        g2d_1005.drawLine(p1_1005.x, p1_1005.y, p2_1005.x,
p2_1005.y);
    }
}

// 2. Gambar Edge Jalur Terbaik (Garis Kuning Tebal)
if (!pathNodes_1005.isEmpty()) {
    g2d_1005.setColor(COLOR_PATH_EDGE_1005);
    g2d_1005.setStroke(new BasicStroke(4));
    for (int i_1005 = 0; i_1005 < pathNodes_1005.size() - 1;
i_1005++) {
        String u_1005 = pathNodes_1005.get(i_1005);
        String v_1005 = pathNodes_1005.get(i_1005 + 1);
        Point p1_1005 = nodePosition_1005.get(u_1005);
        Point p2_1005 = nodePosition_1005.get(v_1005);
        g2d_1005.drawLine(p1_1005.x, p1_1005.y, p2_1005.x,
p2_1005.y);
    }
}

// 3. Gambar Node (Lingkaran + Teks)
for (String node_1005 : vertices_1005) {
    Point p_1005 = nodePosition_1005.get(node_1005);
    int x_1005 = p_1005.x;
    int y_1005 = p_1005.y;

    Color fillColor_1005;
    Color textColor_1005;

    if (node_1005.equals(startSel_1005)) {
        fillColor_1005 = COLOR_START_1005;
        textColor_1005 = COLOR_TEXT_DARK_1005;
    } else if (node_1005.equals(goalSel_1005)) {
        fillColor_1005 = COLOR_GOAL_1005;
        textColor_1005 = COLOR_TEXT_LIGHT_1005;
    } else if (visitedNodes_1005.contains(node_1005)) {
        fillColor_1005 = COLOR_VISITED_1005;
    }
}

```

```

        textColor_1005 = COLOR_TEXT_DARK_1005;
    } else {
        fillColor_1005 = COLOR_UNVISITED_1005;
        textColor_1005 = COLOR_TEXT_DARK_1005;
    }

    // Lingkaran node
    g2d_1005.setColor(fillColor_1005);
    g2d_1005.fillOval(x_1005 - NODE_RADIUS_1005, y_1005 -
NODE_RADIUS_1005, NODE_RADIUS_1005 * 2, NODE_RADIUS_1005 * 2);

    // Border lingkaran
    g2d_1005.setColor(Color.DARK_GRAY);
    g2d_1005.setStroke(new BasicStroke(1.5f));
    g2d_1005.drawOval(x_1005 - NODE_RADIUS_1005, y_1005 -
NODE_RADIUS_1005, NODE_RADIUS_1005 * 2, NODE_RADIUS_1005 * 2);

    // Teks Node (bisa 2 baris jika nama panjang)
    g2d_1005.setColor(textColor_1005);
    g2d_1005.setFont(new Font("Arial", Font.BOLD, 12));
    FontMetrics fm_1005 = g2d_1005.getFontMetrics();
    String shortName_1005 = getShortName_1005(node_1005);

    String[] words_1005 = shortName_1005.split(" ");
    if (words_1005.length > 1) {
        // Bagi jadi 2 baris
        StringBuilder line1_1005 = new StringBuilder();
        StringBuilder line2_1005 = new StringBuilder();
        int mid_1005 = (words_1005.length + 1) / 2;
        for (int i_1005 = 0; i_1005 < words_1005.length;
i_1005++) {
            if (i_1005 < mid_1005) {
                if (line1_1005.length() > 0)
line1_1005.append(" ");
                line1_1005.append(words_1005[i_1005]);
            } else {
                if (line2_1005.length() > 0)
line2_1005.append(" ");
                line2_1005.append(words_1005[i_1005]);
            }
        }
        String l1_1005 = line1_1005.toString();
        String l2_1005 = line2_1005.toString();
        int w1_1005 = fm_1005.stringWidth(l1_1005);
        int w2_1005 = fm_1005.stringWidth(l2_1005);
        g2d_1005.drawString(l1_1005, x_1005 - (w1_1005 / 2),
y_1005 - 2);
        g2d_1005.drawString(l2_1005, x_1005 - (w2_1005 / 2),
y_1005 + 13);
    } else {
        int textWidth_1005 =
fm_1005.stringWidth(shortName_1005);
        g2d_1005.drawString(shortName_1005, x_1005 -
(textWidth_1005 / 2), y_1005 + 5);
    }
}
}

```

```

        // 4. Legenda
        drawLegend_1005(g2d_1005);
    }

    private void drawLegend_1005(Graphics2D g2d_1005) {
        int legendX_1005 = 20;
        int legendY_1005 = getHeight() - 140;
        int boxSize_1005 = 16;
        int gap_1005 = 24;

        Font legendFont_1005 = new Font("Arial", Font.PLAIN, 13);
        g2d_1005.setFont(legendFont_1005);

        String[] labels_1005 = {
            "Start Node", "Goal Node", "Edge pada Jalur",
            "Dikunjungi", "Belum dikunjungi"
        };
        Color[] colors_1005 = {
            COLOR_START_1005, COLOR_GOAL_1005, COLOR_PATH_EDGE_1005,
            COLOR_VISITED_1005, COLOR_UNVISITED_1005
        };

        for (int i_1005 = 0; i_1005 < labels_1005.length; i_1005++)
        {
            int y_1005 = legendY_1005 + i_1005 * gap_1005;

            g2d_1005.setColor(colors_1005[i_1005]);
            g2d_1005.fillRect(legendX_1005, y_1005, boxSize_1005,
boxSize_1005);

            g2d_1005.setColor(Color.DARK_GRAY);
            g2d_1005.drawRect(legendX_1005, y_1005, boxSize_1005,
boxSize_1005);

            g2d_1005.setColor(Color.BLACK);
            g2d_1005.drawString(labels_1005[i_1005], legendX_1005 +
boxSize_1005 + 8, y_1005 + boxSize_1005 - 3);
        }
    }
}

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            PetaPadang_2511531005 frame_1005 = new
PetaPadang_2511531005();
            frame_1005.setVisible(true);
        });
    }
}

```

B. Penjelasan kode Program

Program ini membuat sebuah jendela aplikasi (JFrame) yang menampilkan peta berupa graf (kumpulan node dan garis penghubung/edge). Pengguna dapat memilih Lokasi Awal dan Lokasi Tujuan melalui dropdown, lalu menekan tombol BFS atau DFS untuk mencari jalur. Program akan memvisualisasikan node-node yang dikunjungi selama proses pencarian dan menampilkan jalur akhir yang ditemukan beserta statistiknya di panel teks.

1. Struktur atribut

- Struktur data graf
 - vertices_1005: Menyimpan daftar nama semua lokasi (node).
 - adjList_1005: Adjacency List (berupa HashMap) untuk menyimpan hubungan ketetanggaan antar lokasi.
 - nodePosition_1005: Menyimpan koordinat (X, Y) untuk setiap node agar bisa digambar di posisi yang tetap pada layar.
- Komponen GUI
 - vertices_1005: Menyimpan daftar nama semua lokasi (node).
 - adjList_1005: Adjacency List (berupa HashMap) untuk menyimpan hubungan ketetanggaan antar lokasi.
 - nodePosition_1005: Menyimpan koordinat (X, Y) untuk setiap node agar bisa digambar di posisi yang tetap pada layar.
- Variable status pencarian
 - vertices_1005: Menyimpan daftar nama semua lokasi (node).
 - adjList_1005: Adjacency List (berupa HashMap) untuk menyimpan hubungan ketetanggaan antar lokasi.
 - nodePosition_1005: Menyimpan koordinat (X, Y) untuk setiap node agar bisa digambar di posisi yang tetap pada layar.

2. Inisialisasi graf (initializeGraph_1005)

Fungsi ini untuk membuat peta secara manual dengan cara mendefinisikan 10 lokasi (seperti Unand, Lubuk Buayo, Spendu, dll), memberikan koordinat tetap (X, Y) untuk setiap lokasi agar tata letak graf di layar rapi dan tidak berubah-

ubah, dan menghubungkan lokasi-lokasi tersebut dengan 15 edge (garis) dua arah menggunakan fungsi `addEdge_1005`.

3. Interface GUI(`setupGUI_1005`)

Fungsi ini mengatur tata letak jendela aplikasi menggunakan `BorderLayout`:

- Panel Utara (NORTH): Berisi kontrol pencarian (Label, `ComboBox` Lokasi Awal & Tujuan, serta Tombol BFS, DFS, dan Reset).
- Panel Tengah (CENTER): Berisi `GraphPanel_1005`, yaitu kanvas kosong yang akan di-override untuk menggambar node dan edge graf.
- Panel Selatan (SOUTH): Berisi `JTextArea` di dalam `JScrollPane` untuk menampilkan laporan hasil pencarian (algoritma yang dipakai, node yang dikunjungi, jumlah eksplorasi, dan jalur yang ditemukan).

4. Implementasi algoritma searching

A. Algoritma BFS (`BFS_1005`)

- Menggunakan struktur data Queue (`LinkedList`).
- Memasukkan node awal ke dalam Queue.
- Selama Queue tidak kosong, ambil node paling depan (`poll`), lalu masukkan semua tetangganya yang belum dikunjungi ke dalam Queue.
- Kelebihan BFS: Menjamin menemukan jalur dengan jumlah edge paling sedikit (jalur terpendek secara topologi).
- Tetangga diurutkan (`Collections.sort`) agar penelusurannya konsisten (sesuai abjad).

B. Algoritma DFS (`DFS_1005`)

- Menggunakan struktur data Stack (`Stack`).
- Memasukkan node awal ke dalam Stack.
- Selama Stack tidak kosong, ambil node paling atas (`pop`). Jika belum dikunjungi, tandai sebagai dikunjungi, lalu masukkan semua tetangganya ke dalam Stack.
- Catatan Khusus: Tetangga diurutkan lalu di-reverse (`Collections.reverse`) sebelum di-push ke stack. Ini dilakukan agar saat di-pop, tetangga yang secara abjad paling awal akan keluar dan diproses lebih dulu.

- Karakteristik DFS: Menelusuri graf dengan menilik sedalam mungkin ke satu cabang sebelum berpindah ke cabang lain.

5. actionPerformed

Fungsi ini merespons klik tombol:

- Jika tombol Reset ditekan, semua status pencarian dihapus dan GUI dikosongkan.
- Program memvalidasi bahwa Lokasi Awal dan Tujuan tidak boleh sama.
- Jika tombol BFS atau DFS ditekan, program memanggil metode algoritma yang sesuai dengan lokasi awal dan tujuan yang dipilih dari ComboBox.

6. Visualisasi Graf (kelas Inner GraphPanel_1005)

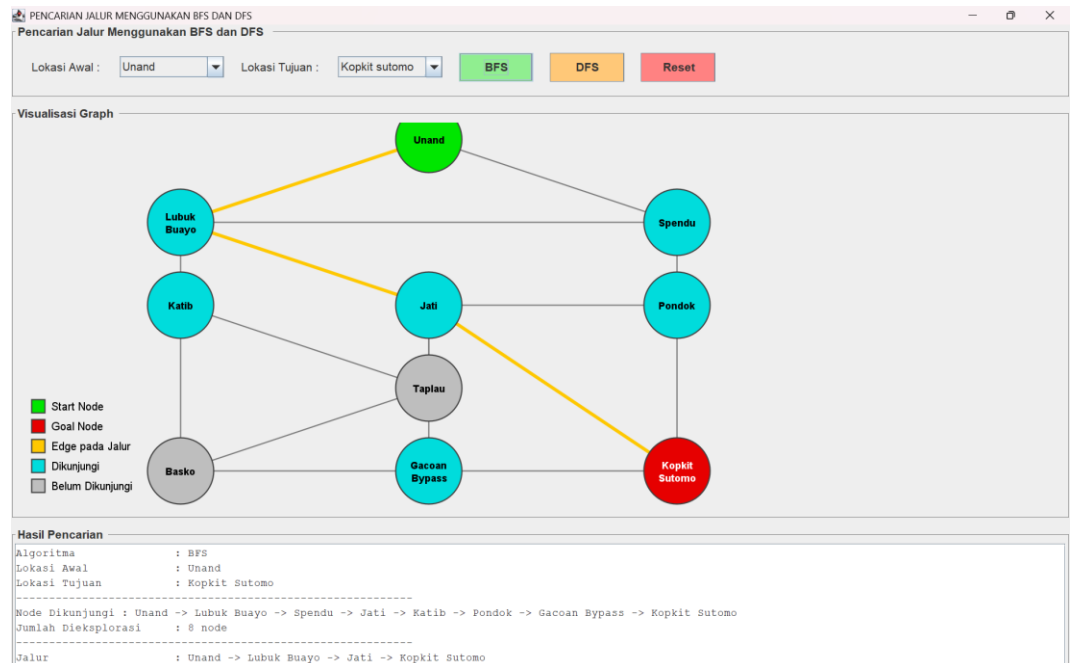
Ini adalah bagian paling kompleks secara visual, di mana metode paintComponent di-override untuk menggambar graf menggunakan Graphics2D:

- Menggambar Edge Dasar: Menarik garis abu-abu tipis untuk semua koneksi antar node.
- Menggambar Jalur Terbaik: Jika ada jalur yang ditemukan (pathNodes_1005 tidak kosong), garis penghubung antar node di jalur tersebut digambar ulang dengan warna kuning dan garis tebal.
- Menggambar Node (Lingkaran):
 - Setiap node digambar sebagai lingkaran dengan radius 40px.
 - Pewarnaan Dinamis:
 - Hijau: Node Awal (Start).
 - Merah: Node Tujuan (Goal).
 - Cyan: Node yang sudah dikunjungi selama pencarian.
 - Abu-abu: Node yang belum dikunjungi.
 - Teks di dalam Node: Program memiliki logika pintar untuk memecah nama lokasi yang panjang (seperti "Gacoan bypass") menjadi 2 baris agar muat di dalam lingkaran, menggunakan perhitungan FontMetrics.
- Menggambar Legenda: Menampilkan kotak warna dan keterangannya di pojok kiri bawah kanvas agar pengguna tahu arti dari setiap warna.

C. Analisis Hasil

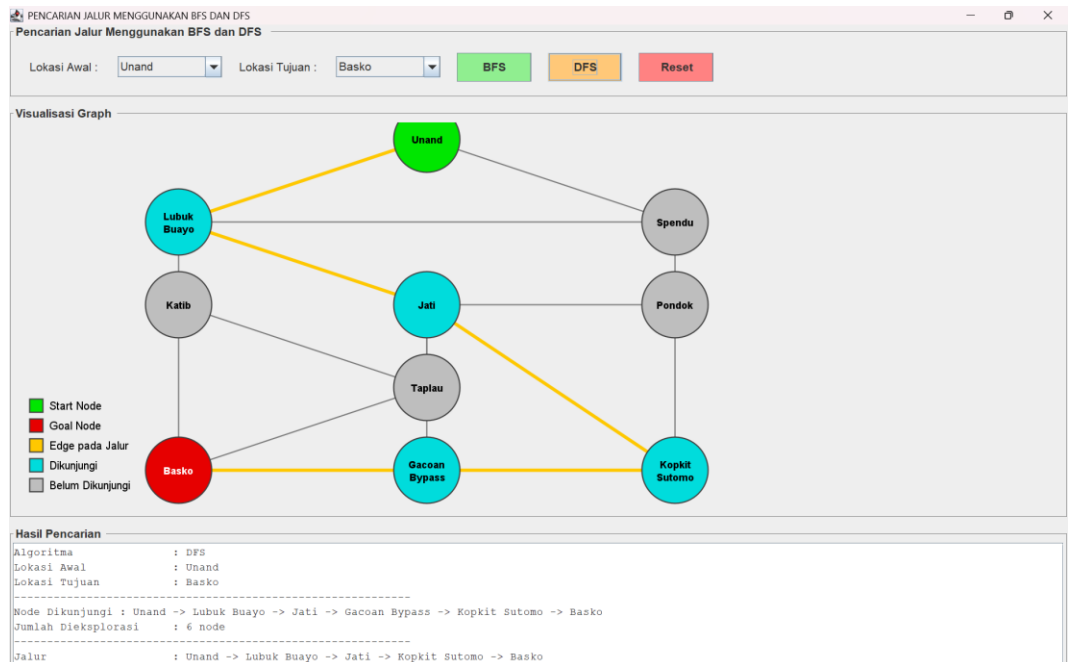
Output program :

1. BFS



Berdasarkan hasil di atas algoritma BFS bekerja secara melebar (level by level). Ia akan mengunjungi semua tetangga terdekat dari node awal sebelum berpindah ke level berikutnya. Program memulai dari Unand. Kemudian mengunjungi tetangga level 1 (Lubuk Buayo, Spendu). Lalu tetangga level 2 (Jati, Katib, Pondok). Terakhir, saat memproses tetangga dari Jati, ia menemukan Gacoan Bypass dan Kopkit Sutomo. Karena Kopkit Sutomo adalah tujuan, pencarian berhenti.

2. DFS



Berdasarkan hasil di atas, algoritma DFS bekerja secara mendalam (depth-first). Ia akan menuruni satu cabang sejauh mungkin sampai mentok (dead end) atau menemukan tujuan, baru kemudian melakukan backtracking (kembali) untuk mencoba cabang lain. DFS hanya mengunjungi 6 node, jauh lebih sedikit dibanding BFS di kasus sebelumnya, karena ia langsung "menembak" ke arah tujuan melalui cabang tengah.